Wow, that’s scary, isn’t it? Now, if there only was a way to avoid it!

Many companies, even tech giants, in their early days (and at the dawn of the Internet age), had to experience first-hand the troubles of running and growing a website smoothly. That is, till they found ways to adapt and cope with it.

One of the best ways to ease the load on a database is indeed avoiding to compute expensive results twice in a short time.

There are many other strategies orthogonal to this, from *sharding*[5] to loosening the consistency constraint (moving to *eventual consistency*[6]), that are needed or at least helpful in order to achieve good, stable performance on a web application. This is obviously not the right place to present them however, as you can imagine – but the literature on scalability is rich, and if you are going to work on the fields, we definitely recommend you take a look at some of the amazing books or web guides that have been published[7].

In this chapter we are going to focus, instead, on caching. To narrow down a bit our scope, and hopefully make our example clearer, let’s consider a plausible situation where you could actually use some caching.

Imagine you have this aggregator service that gathers news about companies from social networks, and provides some sort of insight on them, for example if people are talking about a company in a positive or negative way (what’s called